DSA PROBLEM NOTES

Majority Element • LeetCode #169

1. PROBLEM IN MY WORDS

Given an integer array nums, find the element that appears more than n / 2 times. This element is called the majority element.

2. EXAMPLE

Input	nums = [2, 2, 1, 1, 1, 2, 2]
Output	2
Why?	2 appears 4 times, and 4 > 7/2.

3. BRUTE FORCE

Use a HashMap to count the frequency of every number. Then return the number whose frequency is greater than n / 2.

Time: O(n)
Space: O(n)

4. PATTERN

Boyer-Moore Voting Algorithm

Why?
The problem guarantees that a majority element exists. Because it occurs more than half the time, it can survive cancellation against all other elements.

5. CORE IDEA

Keep two variables:

candidate → current possible majority element
count → its current vote balance

For every number:
• If count = 0 → make the current number the candidate.
• If current number = candidate → count++.
• Otherwise → count--.

The majority element has more occurrences than all other elements combined, so after cancellation it remains as the final candidate.

6. DRY RUN

nums = [2, 2, 1, 1, 1, 2, 2]

Start: candidate = 2, count = 0

2 → count = 1
2 → count = 2
1 → count = 1
1 → count = 0
1 → count = 1, candidate = 1
2 → count = 0
2 → count = 1, candidate = 2

Final candidate = 2

7. CODE

```
class Solution {
    public int majorityElement(int[] nums) {
        int candidate = 0;
        int count = 0;

        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }
            if (num == candidate) {
                count++;
            } else {
                count--;
            }
        }

        return candidate;
    }
}
```

8. COMPLEXITY

Time Complexity: O(n)
Space Complexity: O(1)

We scan the array once and use only two variables.

9. MISTAKES ⚠

- Don't confuse majority element with the largest element.
- Majority means frequency > n/2, not simply the most frequent element.
- Boyer-Moore works directly here because the problem guarantees a majority element.
- If the problem does not guarantee a majority element, verify the candidate when necessary.

10. RECOGNITION CLUE

Element appears more than n/2 times → think Boyer-Moore Voting Algorithm.

Main clue: one element has more votes than all other elements combined.

11. RELATED PROBLEMS

- Majority Element II (elements appearing more than n/3 times)
- Find the majority/frequent element using HashMap
- Problems involving frequency counting and voting/cancellation